DVWA Security Notes

Command Execution & SQL Injection — A Developer's Perspective on How and Why These Vulnerabilities Work

PART 1 | Command Execution (DVWA)

Goal: understand why command injection happens by reading the vulnerable PHP code itself, rather than only exploiting it from the outside.

Low Security

LOW

Source code

```
$target = $_REQUEST['ip'];
$cmd = shell_exec('ping -c 3 ' . $target);
```

Key observation

The application directly takes user input and concatenates it into a shell command, with no validation at all.

Example exploit

```
Input: 127.0.0.1;whoami

Built command:
ping -c 3 127.0.0.1;whoami
```

Because **shell_exec()** passes the entire string through the Linux shell, the shell interprets **;** as a command separator. Linux therefore executes two commands in sequence: first **ping -c 3 127.0.0.1**, then **whoami** — which returns **www-data**.

■ *Why we knew it was vulnerable: shell_exec() executes shell commands, $target comes directly from user input, and the input is concatenated without validation — so shell metacharacters can be interpreted as syntax rather than data.*

Operators tested

Operator	Payload	Meaning	Result
;	`127.0.0.1;whoami`	Execute ping, then execute whoami unconditionally.	Worked — returned www-data
&&	`127.0.0.1&&whoami`	Execute whoami only if ping succeeds.	Worked — ping succeeded

Operator	Payload	Meaning	Result
|	`127.0.0.1|whoami`	Pipe ping's output into whoami.	Worked, but whoami ignores stdin and just reports www-data directly — which is why only that appeared, not the ping output.

Medium Security

MEDIUM

New code

```
$substitutions = array(
'&&' => '',
';' => '',
);

$target = str_replace(...);
```

Before executing the command, the application strips out **;** and **&&**. The input `127.0.0.1;whoami` becomes `127.0.0.1whoami`, and no injection occurs.

However, `127.0.0.1|whoami` still works, because **|** is not on the blacklist.

■ *Lesson: blacklist security is weak. Developers often forget dangerous operators, and attackers simply use one that wasn't blocked.*

High Security

HIGH

Main protection

```
$octet = explode(".", $target);

// then, for every octet:
is_numeric()
```

The IP address is split into its four octets, and each one must pass **is_numeric()**. For a normal address like `127.0.0.1`, the octets are 127, 0, 0, and 1 — all numeric.

For a payload like `127.0.0.1;whoami`, the last octet becomes `1;whoami`, which is **not** numeric. Validation fails before **shell_exec()** is ever reached, and the app returns *"ERROR: You have entered an invalid IP"*.

■ *Lesson: this is whitelist validation — only valid IPv4 addresses are accepted. It is far stronger than a blacklist because it defines what's allowed, not what's forbidden.*

Security Level Comparison — Command Execution

Level	Defense	Outcome
Low	No validation; direct shell execution.	Completely vulnerable.
Medium	Blacklist removes a few dangerous operators.	Easy to bypass with an operator not on the list.
High	Whitelist: only valid IP addresses accepted.	shell_exec() never receives malicious input.

PART 2 | SQL Injection (DVWA)

Goal: understand SQL Injection by reading the vulnerable PHP code.

Low Security

LOW

Vulnerable code

```
$id = $_GET['id'];

$getid =
"SELECT first_name,last_name
FROM users
WHERE user_id='$id'";
```

The application inserts user input directly into the SQL query string.

Example exploit

```
Input: 1' OR '1'='1

Query becomes:
SELECT ...
WHERE user_id='1' OR '1'='1'
```

Since `'1'='1'` is always true, the query returns **every row** in the table. This happens because of direct string concatenation with no validation and no prepared statements.

Quote-based payloads

Used when the application places user input inside quotes, e.g. `WHERE user_id='$id'`. The payload's leading quote closes the original string, letting the rest of the input be interpreted as new SQL: `1' OR '1'='1`.

Medium Security

MEDIUM

New code

```
$id = mysql_real_escape_string($id);

// Query:
WHERE user_id = $id
```

mysql_real_escape_string() escapes special SQL characters by adding a backslash, so a payload such as `1' OR '1'='1` becomes roughly `1\' OR \'1\'=\'1`. The quotes are now treated as ordinary characters instead of SQL syntax, which blocks many classic quote-based payloads.

■ *Removing quotes from the query itself is not what makes this safer — the important change is escaping the special characters. Even so, escaping is not considered the best modern solution.*

Numeric-based payloads

Used when the application expects a number and does **not** wrap input in quotes, e.g. `WHERE user_id = $id`. A payload like `1 OR 1=1` needs no quotes at all.

■ *Rule of thumb — if the input is wrapped in quotes in the query, use a quote-based payload; if it isn't, use a numeric payload.*

High Security

HIGH

New protections

```
$id = stripslashes($id);
$id = mysql_real_escape_string($id);

if (is_numeric($id))
// proceed with query
```

Only numeric input is accepted. `1` is allowed, while `1' OR '1'='1` is rejected outright — since it isn't numeric, the application never even executes the SQL query.

Escaping special characters

Escaping prevents characters such as the single quote from being interpreted as SQL syntax. `1' OR '1'='1` becomes `1\' OR \'1\'=\'1` after escaping — the SQL parser no longer treats the quotes as string delimiters.

Security Level Comparison — SQL Injection

Level	Defense	Outcome
Low	Direct SQL string concatenation.	Fully vulnerable.
Medium	Escapes special characters.	Blocks many classic quote-based injections; still not modern security.
High	Escaping + strict numeric validation.	Only numeric IDs reach the database; stronger than Medium.

Best Practice — Real Applications

Neither escaping nor blacklisting is considered the best solution today. Modern applications use **prepared statements** (parameterized queries):

```
SELECT first_name, last_name
FROM users
WHERE user_id = ?
```

■ *The database always treats user input as DATA, never as SQL code. This completely separates code from user input and is the standard defense against SQL Injection.*

Notes compiled from DVWA (Damn Vulnerable Web Application) — an intentionally vulnerable app used for defensive security education and training.